



IIC2343 - Arquitectura de Computadores (I/2026)

Guía de ejercicios: *pipeline*

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl), José Mendoza (jfmendoza@uc.cl)

Pregunta 1: Conceptos

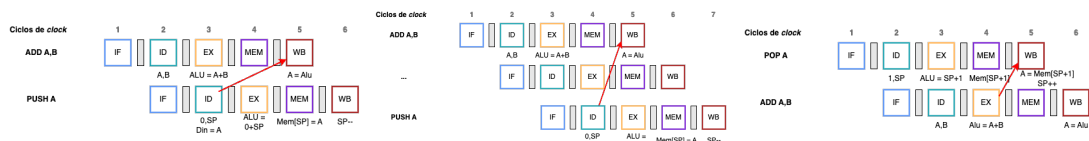
- (a) La arquitectura del computador básico con *pipeline* no permite la ejecución de la instrucción de salto JGE. Explique por qué no es posible su ejecución y describa qué modificaciones habría que realizar para implementarla. Puede asumir que la ALU cuenta con todos los circuitos internos del computador básico sin *pipeline*.

Solución: No se puede ejecutar la instrucción JGE por la **falta de la *flag* N**. El salto JGE se realiza si, y solo si $A \geq B$, lo que se evalúa con la *flag* N igual a 0. Para su ejecución, asumiendo que se cuenta con todos los circuitos internos de la ALU del computador básico sin *pipeline*, se pueden aplicar las siguientes modificaciones:

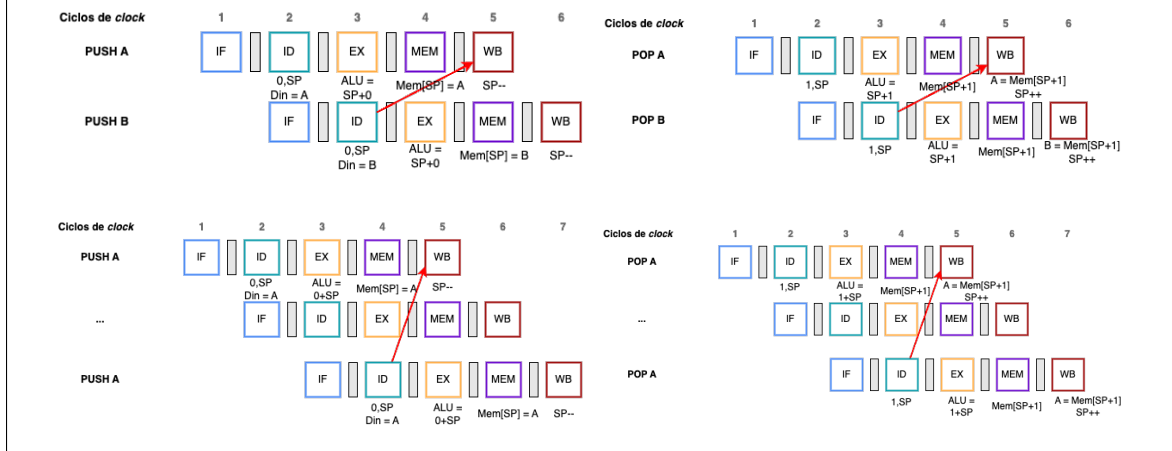
- Conectar la salida de la *flag* N de la ALU con el registro intermedio EX/MEM.
- Transmitir la señal N a la *Jump Unit*.
- Modificar la *Jump Unit* para que evalúe la condición N igual a 0 en caso de ejecutar una instrucción JGE.

- (b) Suponga que se modifica el computador básico con *pipeline* para reincorporar las instrucciones de manejo de *stack*: PUSH A/B y POP A/B. Indique dos casos **distintos** de *hazard* de datos que podrían surgir a raíz de esta modificación, explicando en cada ejemplo la etapa en la que surge la dependencia. Se considerarán “casos distintos” si se detectan con combinaciones de señales distintas.

Solución: Como PUSH A y PUSH B corresponden a instrucciones de escritura de memoria, mientras que POP A y POP B son de lectura de memoria, entonces pueden generar los mismos tipos de *hazards* de estos tipos de instrucción. A continuación, se muestran tres casos distintos:



Por otra parte, si asumiéramos que el *stack pointer* se actualiza en la etapa **Writeback**, podrían surgir otros cuatro casos distintos:



- (c) Suponga que en el computador básico con *pipeline* corre un programa de 100 instrucciones. Si un 10% de ellas son de salto, y en un 75% de ellos el salto **no se realiza**, ¿cuál es el número aproximado de ciclos perdidos por *flushing*? Justifique a través de cálculos.

Solución: En el computador básico con *pipeline*, la unidad predictora de saltos asume que estos nunca se realizan. Por lo tanto, cada vez que se realiza un salto, se pierden tres ciclos por *flushing*. Luego, considerando la información otorgada, se tiene el siguiente número aproximado de ciclos perdidos:

$$3_{\text{Ciclos}} \times (100_{\text{Instrucciones}} \times 0,1_{\text{Saltos}} \times 0,25_{\text{Saltos que se realizan}}) = 7,5_{\text{Ciclos}}$$

Se otorga **1 pto.** por señalar la cantidad correcta de ciclos perdidos por salto, y **1 pto.** por calcular correctamente el número perdido de ciclos en total con los datos entregados, siempre que se respalden con cálculos.

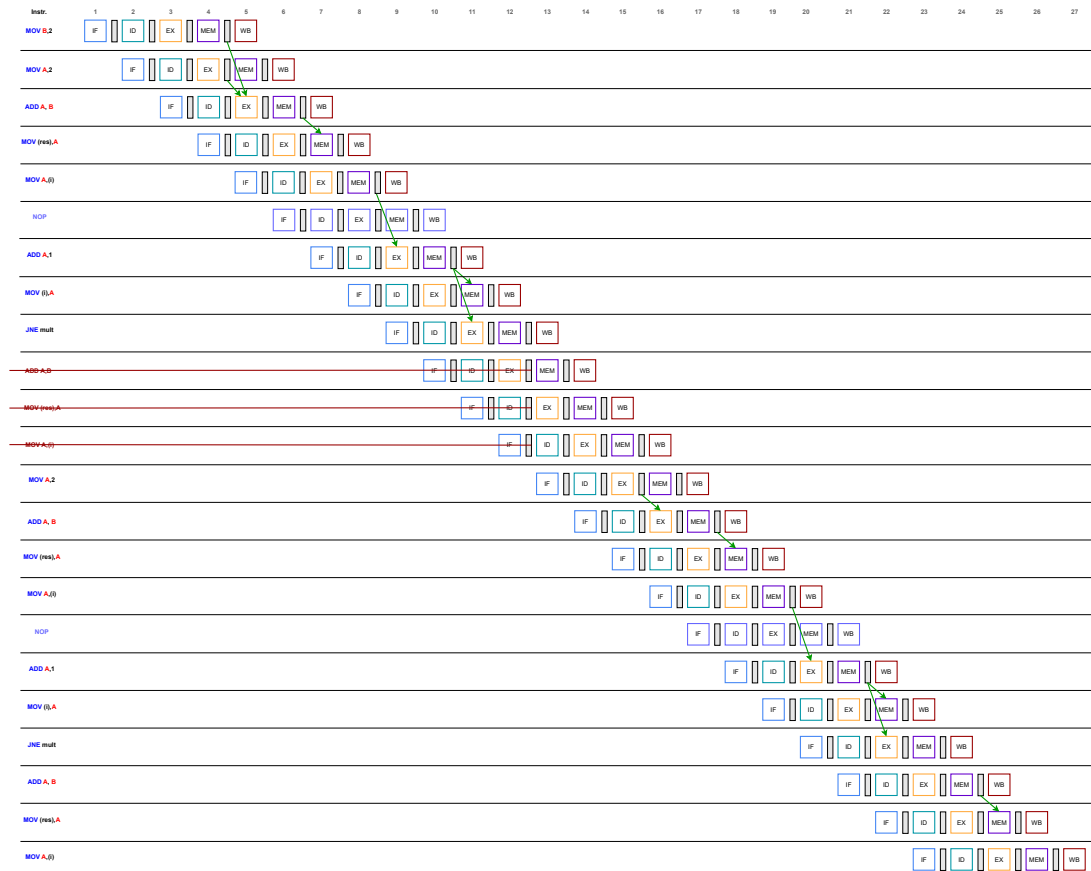
Pregunta 2: Ejecución

(a) A partir del siguiente programa del computador básico con *pipeline*:

```
DATA:
    res 0
    i   0
CODE:
    MOV B,2
    mult:
        MOV A,2
        ADD A,B
        MOV (res),A
        MOV A,(i)
        ADD A,1
        MOV (i),A
        JNE mult
        ADD A,B
        MOV (res),A
        MOV A,(i)
```

Determine el número de ciclos que demora su ejecución detallando en un diagrama los estados del *pipeline* por instrucción. Asuma que el manejo de *stalling* es por *software* a través de la instrucción NOP y que la unidad predictora de saltos asume que estos nunca se realizan, independiente de que estos sean condicionales o incondicionales. Indique en el diagrama adjunto al enunciado cuando ocurre *forwarding* (con flechas desde el registro hacia la etapa que corresponda), *stalling* y *flushing* (con tachado en las instrucciones *flushheadas*).

Solución: A continuación, se muestra el *pipeline* resultante de la ejecución del programa.



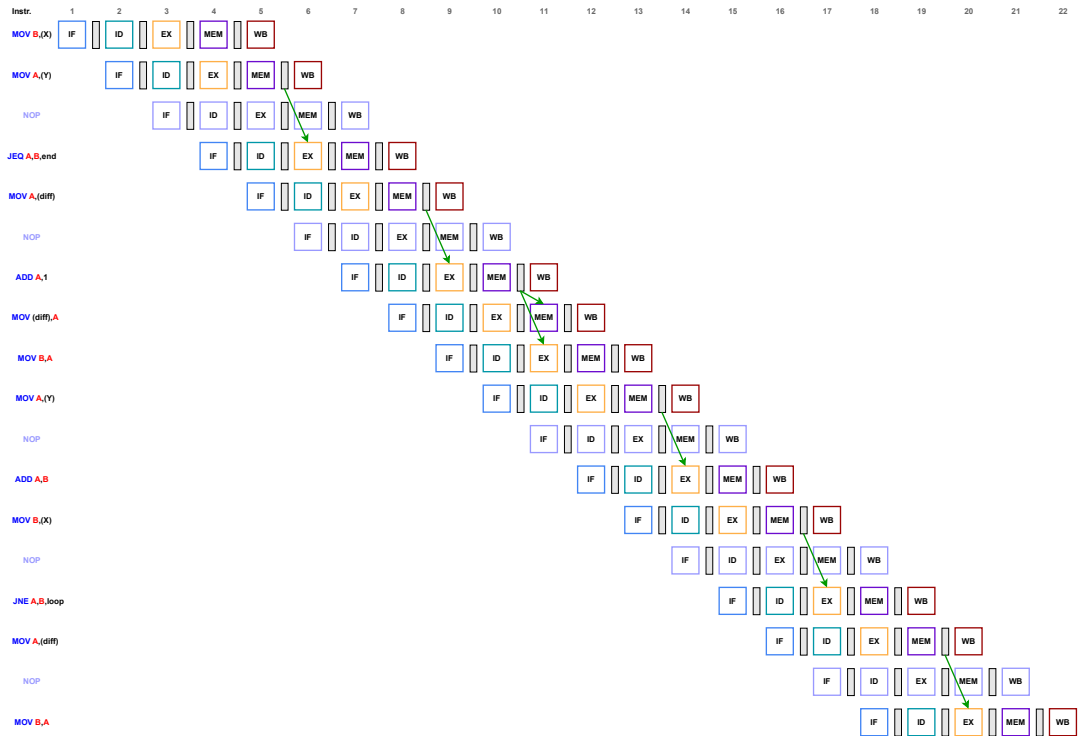
Del *pipeline* anterior se deduce que el programa termina de correr después de 27 ciclos.

(b) A partir del siguiente programa del computador básico con *pipeline*:

```
DATA:
  X    7
  Y    6
  diff 0
CODE:
  MOV B, (X)
  MOV A, (Y)
  JEQ A, B, end
loop:
  MOV A, (diff)
  ADD A, 1
  MOV (diff), A
  MOV B, A
  MOV A, (Y)
  ADD A, B
  MOV B, (X)
  JNE A, B, loop
end:
  MOV A, (diff)
  MOV B, A
```

Determine el número de ciclos que demora su ejecución detallando en un diagrama los estados del *pipeline* por instrucción. Asuma que el manejo de *stalling* es por *software* a través de la instrucción NOP y que la unidad predictora de saltos asume que estos nunca se realizan, independiente de que estos sean condicionales o incondicionales. Indique en el diagrama adjunto al enunciado cuando ocurre *forwarding* (con flechas desde el registro hacia la etapa que corresponda), *stalling* y *flushing* (con tachado en las instrucciones *flushheadas*).

Solución: A continuación, se muestra el *pipeline* resultante de la ejecución del programa.



Del *pipeline* anterior se deduce que el programa termina de correr después de 22 ciclos.